

CO4 – AT2

Industry Problem Solving Task

REPORT:

Travelling Salesman Problem (TSP) – Logistics Optimization Using Dynamic Programming

Aim

To solve the Travelling Salesman Problem for 15 cities using the Dynamic Programming based Held-Karp algorithm and determine the minimum travel cost required to visit every city exactly once and return to the starting depot.

Problem Statement

A delivery company needs to visit 15 cities. The vehicle must:

- Start from the depot.
- Visit every city exactly once.
- Return to the depot.
- Minimize the total travel cost.

This is a Travelling Salesman Problem (TSP).

TSP is an important optimization problem in logistics and transportation because it can be used to determine efficient routes for delivery vehicles.

Input

The input consists of:

- Number of cities: **15**
- A cost/distance matrix representing the travel cost between every pair of cities.
- City 0 is considered the starting depot.

For example:

City 0 $\rightarrow$ Depot

City 1 $\rightarrow$ City 1

City 2 $\rightarrow$ City 2

City 14 $\rightarrow$ City 14

Output

The algorithm produces:

Minimum Travel Cost = minimum possible tour cost

The tour starts at city 0, visits all 14 other cities exactly once, and returns to city 0.

Dynamic Programming Approach

The Held-Karp algorithm solves TSP using Dynamic Programming.

Instead of checking every complete route independently, it stores the best solution for smaller subsets of cities and reuses those results.

The important idea is:

Find the minimum cost of reaching a particular city after visiting a particular subset of cities.

State Representation

Let:

$DP[S][i]$

represent the minimum cost of starting from the depot, visiting every city in subset S , and ending at city i .

Where:

- S = Set of visited cities.
- i = Last visited city.

- $DP[S][i]$ = Minimum travel cost for that state.

For example:

$$S = \{1, 2, 3\}$$

$$i = 3$$

means the route starts from the depot, visits cities 1, 2 and 3, and ends at city 3.

Bitmask Representation

A bitmask is used to represent the visited cities.

For 4 cities:

City: 1 2 3

Bit: 0 1 2

If cities 1 and 3 are visited:

101

The bitmask makes it efficient to determine which cities have already been visited.

For 15 cities, the number of subsets is:

$$2^{14} = 16,384$$

The starting depot is excluded from the subset because it is always the starting point.

Recurrence Relation

The Held-Karp recurrence is:

$$DP[S][i] = \min(DP[S - \{i\}][j] + \text{cost}[j][i])$$

where:

$$j \in S - \{i\}$$

This means that to reach city i , the algorithm considers every possible previous city j and chooses the route with minimum cost.

Base Case

For a direct trip from the depot to city i:

$$DP[\{i\}][i] = \text{cost}[0][i]$$

Final Answer

After all cities have been visited, the salesman must return to the depot.

Therefore:

$$\text{Answer} = \min(DP[\text{All Cities}][i] + \text{cost}[i][0])$$

The final return trip from city i to the depot is included.

Algorithm

1. Represent cities using a bitmask.
2. Create a DP table.
3. Initialize the cost of travelling directly from the depot to every city.
4. Generate subsets of cities.
5. For each subset, consider every possible last city.
6. Try every possible previous city.
7. Select the minimum-cost route.
8. Repeat until all cities are included.
9. Add the cost of returning to the depot.
10. Select the minimum final cost.

Pseudocode

TSP(cost, n)

 Create DP table

 FOR each city i from 1 to n-1

$DP[\{i\}][i] = cost[0][i]$

FOR each subset S of cities

FOR each city i in S

$DP[S][i] = INFINITY$

FOR each city j in S, $j \neq i$

$DP[S][i] =$

$MIN(DP[S][i],$

$DP[S - \{i\}][j] + cost[j][i])$

fullSet = all cities except depot

answer = INFINITY

FOR each city i

answer =

$MIN(answer,$

$DP[fullSet][i] + cost[i][0])$

RETURN answer

Example

Suppose the route is:

Depot $\rightarrow$ C1 $\rightarrow$ C2 $\rightarrow$ C3 $\rightarrow$... $\rightarrow$ C14 $\rightarrow$ Depot

The algorithm does not directly test only complete routes.

Instead, it first calculates smaller routes such as:

Depot $\rightarrow$ C1

Depot $\rightarrow$ C2

Depot $\rightarrow$ C1 $\rightarrow$ C2

Depot $\rightarrow$ C2 $\rightarrow$ C1

Depot $\rightarrow$ C1 $\rightarrow$ C2 $\rightarrow$ C3

The best results are stored in the DP table and reused to build larger routes.

Why Dynamic Programming is Suitable

A brute-force TSP algorithm checks approximately:

$(n - 1)!$

possible routes.

For 15 cities:

$14! = 87,178,291,200$

possible routes.

Checking all these routes is expensive.

The Held-Karp algorithm reduces the complexity to:

$O(n^2 \times 2^n)$

which is significantly better than brute force for smaller values of n .

Time Complexity

The Held-Karp algorithm has:

Time Complexity = $O(n^2 \times 2^n)$

For $n = 15$:

$O(15^2 \times 2^{15})$

The algorithm is still exponential, but much more practical than checking all possible permutations.

Space Complexity

The DP table requires approximately:

$O(n \times 2^n)$

space.

Therefore:

Space Complexity = $O(n \times 2^n)$

For 15 cities:

$$15 \times 2^{14}$$

$$= 15 \times 16,384$$

$$= 245,760 \text{ DP states}$$

The exact memory consumption depends on the data type and implementation.

Scalability

The major limitation of Held-Karp is exponential growth.

As the number of cities increases:

Cities	Number of subsets
--------	-------------------

10	$2^{10} = 1,024$
----	------------------

15	$2^{15} = 32,768$
----	-------------------

20	$2^{20} = 1,048,576$
----	----------------------

25	$2^{25} = 33,554,432$
----	-----------------------

30	$2^{30} = 1,073,741,824$
----	--------------------------

Therefore, memory and computation requirements increase rapidly.

Tools for Implementation

Suitable tools include:

C/C++

Useful for implementing the Held-Karp algorithm efficiently because of its speed and memory control.

Python

Useful for prototyping and experimenting with TSP algorithms. Libraries such as NumPy can help with numerical operations.

MATLAB

Useful for algorithm development, visualization and optimization experiments.

OR-Tools

Google OR-Tools can be used for practical vehicle-routing and optimization problems.

IDEs

The program can be developed using:

- Visual Studio Code
- Dev-C++
- Code::Blocks
- GCC/MinGW

Result

The Travelling Salesman Problem for 15 cities was successfully formulated and solved using the Held-Karp Dynamic Programming algorithm.

The algorithm:

- Visits every city exactly once.
- Starts from the depot.
- Returns to the depot.
- Minimizes the total travel cost.